

Traivo - Specifikation för import, matchning och klustrering av objekt

Datum: 2026-06-08
Källa: Hemköp-importfil + UI-skärmdumpar från Traivo
Syfte: Komplett teknisk specifikation för att implementera importflödet i Replit (eller annan plattform).

1 Översikt

Traivo tillhandahåller ett importflöde där en kund laddar upp en Excel-/CSV-fil med objektdata. Systemet:

1. **Läser in filen** – identifierar kolumner.
2. **Matchar kolumner** mot systemfält (standardfält, adressfält, kontaktfält, metadata).
3. **Validerar datatyper** – text, tal, datum, GPS, boolesk, obligatoriska fält m.m.
4. **Skapar/uppdaterar objekt** och bygger en trädstruktur (klustrering) med hjälp av interimsnummer och interimföräldranummer.

2 Datastruktur i importfilen (Hemköp-exempel)

2.1 Filformat

Egenskap	Värde
Format	<code>.xlsx</code> (stöd för <code>.csv</code> bör finnas)
Blad	<code>Blad1</code> (enda bladet)
Rader totalt	894 (3 header-rader + 891 datarader)
Kolumner	19 (A-S)

2.2 Header-struktur (rad 1-3)

Rad	Innehåll	Syfte
Rad 1	Systemfältnamn (Systemnummer , Objektnamn , externt_id , adress.gata ...)	Matchning mot Traivos API-fält
Rad 2	Beskrivningstext per kolumn	Tooltip / dokumentation
Rad 3	Kundens egna rubriker (Objektnamn , Butksnummer , Postadress ...)	Användarens perspektiv

2.3 Kolumndefinition

#	System-fält (rad 1)	Kundens rubrik (rad 3)	Datatyp	Matchas mot	Obligatorisk	Beskrivning
A	Systemnummer	Finns ej ännu	Text/ID	system_id	Nej	Traivos unika ID. Tomt = nytt objekt; ifyllt = uppdatera
B	System-föräldranummer	Finns ej ännu	Text/ID	system_parent_id	Nej	Peka på befintlig förälder via systemnummer
C	Objekt-namn	Objekt-namn	Text	name	Ja	Objektets namn – obligatoriskt på varje rad
D	Interimsnummer	Intrum-nummer	Text/ID	inter-im_id	Nej*	Temporärt löpnummer för att binda ihop hierarkin
E	Interim-föräldranummer	Överordnat objekt	Text/ID	inter-im_parent_id	Nej	Pekar på en annan rads interimsnummer; tomt = rotnivå
F	(ej system-fält)	Objekt typ	Text/lista	metadata.typ	Nej	Organisation/kund, Kedja, Destrikt, Butik, Pantkärl ...
G	extern-ternt_id	Butksnummer	Text/ID	external_id	Nej	Kundens egen referens (ex.

#	System-fält (rad 1)	Kundens rubrik (rad 3)	Datatyp	Matchas mot	Obligatorisk	Beskrivning
						HK21Tingsryd)
H	ad-ress.gata	Postadress	Text/adress	ad-dress.street	Nej	Gatunamn
I	ad-ress.gatunummer	(ej ifylld)	Text	ad-dress.street_number	Nej	Gatunummer
J	ad-ress.postnummer	Postnr	Text	ad-dress.postal_code	Nej	Postnummer
K	ad-ress.ort	Ort	Text	ad-dress.city	Nej	Ort
L	(ej system-fält)	Region	Text	metadata.region	Nej	Distriktets namn (för referens)
M	kontakt-person.namn	Namn	Text	contact.name	Nej	Kontaktpersons namn
N	kontakt-person.titel	Titel	Text	contact.title	Nej	Titel/roll (ex. BC)
O	kontakt-person.telefon	Telefon	Text	contact.phone	Nej	Telefonnummer
P	(ej system-fält)	E-post	Text/e-post	contact.email	Nej	E-postadress
Q	(ej system-fält)	Antal	Tal	metadata.antal	Nej	Antal enheter (ex. 41, 51)

#	System-fält (rad 1)	Kundens rubrik (rad 3)	Datotyp	Matchas mot	Obligatorisk	Beskrivning
R	(ej system-fält)	Tömningsdag	Tal	metadata.t ömningsdag	Nej	Veckodag/frekvens
S	(ej system-fält)	Färg	Text	metadata.f ärg	Nej	Färgkod (Grön, Blå, Brun)

*Interimsnummer är tekniskt valfritt men krävs för att bygga hierarkin vid nyimport.

3 Objekthierarki och klustrering

3.1 Hierarkinivåer i Hemköp-filen

```

Nivå 1: Organisation/kund (1 st)      interim 1000
├─ Nivå 2: Kedja (1 st)              interim 2000, förälder → 1000
│   └─ Nivå 3: Destrikt (10 st)      interim 3000–12000, förälder → 2000
│       └─ Nivå 4: Butik (220 st)     interim 1–220, förälder → 3–12
│           └─ Nivå 5: Utrustning (659 st)
│               - Pantkärl (220 st)
│               - Matavfallskärl (220 st)
│               - Kundvagnsgarage(219 st)

```

3.2 Regler för hierarkiuppbyggnad

Regel	Beskrivning
Rotnivå	Interimförälldranummer är tomt eller "-" → objektet placeras på toppnivå
Intern förälder	Interimförälldranummer pekar på en annan rads Interimsnummer i samma fil
Befintlig förälder	Systemförälldranummer pekar på ett redan befintligt objekt i Traivo
Utrustningskoppling	Utrustningsrader (Pantkärl, Matavfallskärl, Kundvagnsgarage) delar Interimsnummer med sin butik men har tomt Interimförälldranummer – kopplingen sker genom att gruppera rader med samma interimsnummer
Prioritet	Systemnummer > externt_id > Interimsnummer vid matchning mot befintliga objekt

3.3 Nycklar för identifiering

Nyckeltyp	Fält	Betydelse
Systemnummer	Systemnummer	Objektet finns redan i Traivo → uppdatera
Interimsnummer	Interimsnummer	Nytt objekt i denna importfil → skapa
externt_id	externt_id	Kundens egen referens → matcha mot befintligt
__EMPTY	Speciellt fält	Markerar att kolumnen ska lämnas tom / ignoreras

4 Fälttyper och matchningslogik

4.1 Systemfältkategorier

Baserat på UI-modalen "Matcha data" finns följande fältkategorier:

Standardfält

Fältnamn	API-nyckel	Beskrivning
Systemnummer	system_id	Traivos unika ID - ifyllt = uppdatera
Systemföräldranummer	system_parent_id	Peka mot befintlig förälder
Objektnamn	name	Obligatoriskt - namn på objektet
Interimsnummer	interim_id	Temporärt ID för hierarki
Interimföräldranummer	interim_parent_id	Temporär förälderreferens
__EMPTY	-	Ignorera kolumnen

Metadatafält (kundens egna)

Fältnamn	API-nyckel	Beskrivning
externt_id	external_id	Kundens egna referensnummer
metadata.typ	metadata.typ	Objekttyp (fritext/lista)
metadata.volym	metadata.volym	Antal/volym (tal/text)
metadata.bil	metadata.bil	Fordon/utförrarresurs
metadata.tidsåtgång	metadata.tidsåtgång	Tidsåtgång
metadata.*	metadata.*	Valfria kunddefinierade fält

Adressfält (punktnotation → sammansatt objekt)

Fältnamn	API-nyckel	Grupperas till
adress.gata	address.street	address
adress.gatunummer	address.street_number	address
adress.postnummer	address.postal_code	address
adress.ort	address.city	address
Leveransadress	address.full	address (fullständig adress)

Kontaktfält (punktnotation → sammansatt objekt)

Fältnamn	API-nyckel	Grupperas till
kontaktperson.namn	contact.name	contact
kontaktperson.titel	contact.title	contact
kontaktperson.telefon	contact.phone	contact

4.2 Matchningsalgoritm (kolumnidentifiering)

För varje kolumn i importfilen:

1. Läs rubrik från rad 1 (systemfältnamn)
2. Försök exakt matchning mot kända systemfält
3. Om ingen **match** ☐ försök fuzzy **match** (Levenshtein / normaliserad text)
4. Om ingen **match** ☐ presentera kolumnen som omatchad för manuell koppling
5. Användaren kan välja fält från "Matcha data"-dialogen
6. Kolumner markerade som "Matchad" (☒ ☐) är klara

5 Valideringsregler

5.1 Per datatyp

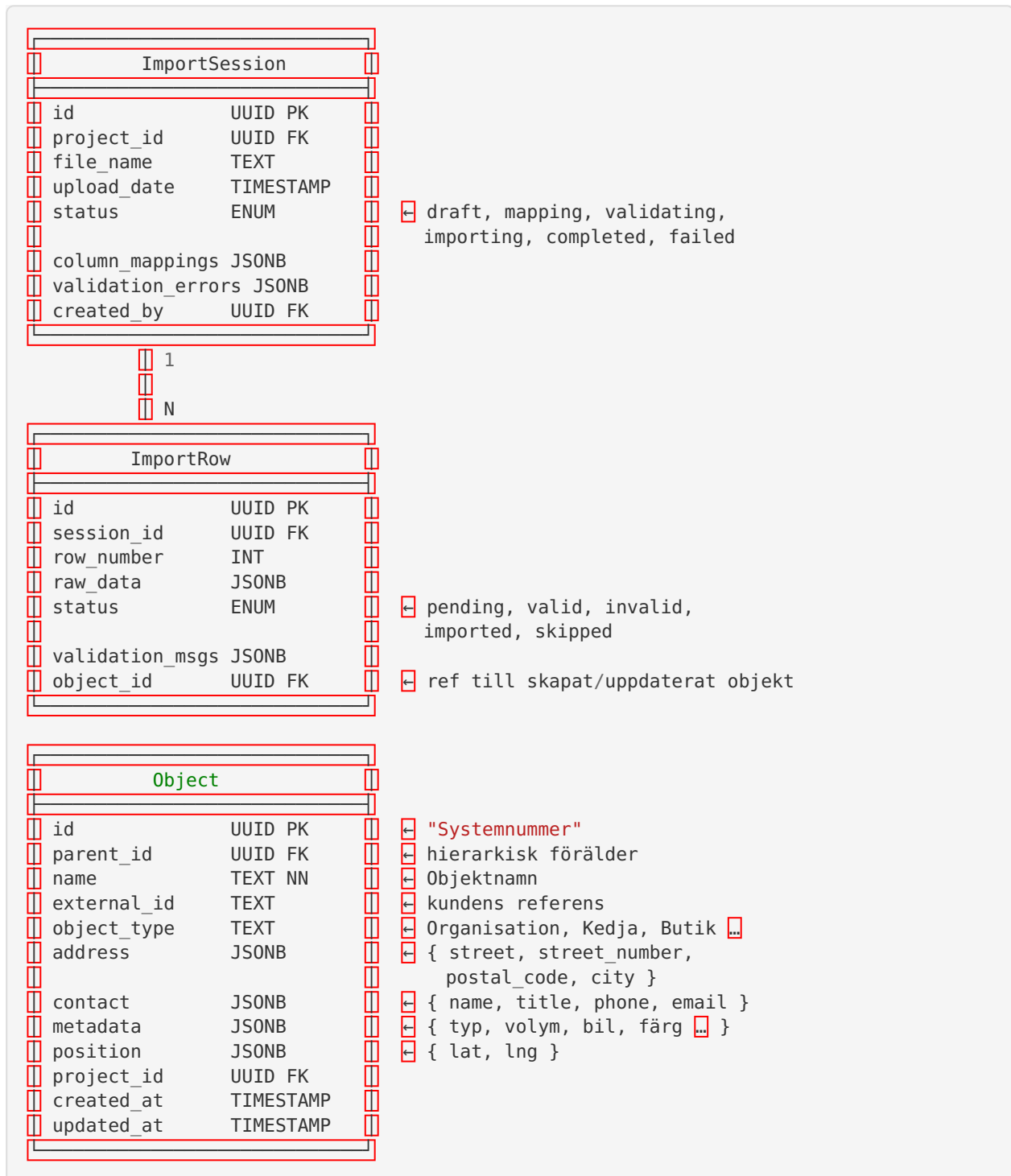
Datatyp	Validering	Felhantering
Text	Icke-tomt om obligatoriskt	Stoppa rad, rapportera
Text/ID	Alfanumerisk, bindestreck, understreck	Varning
Tal	Numeriskt värde (heltal eller decimal)	Hoppa över / nollställ
Decimal (GPS)	WGS84-format (ex. 59.3293, 18.0686)	Varning
Datum	ISO 8601 eller lokalt format	Parsningsfel → manuell fix
E-post	Regex-validering <code>*@*.*</code>	Varning
Telefon	Siffror, mellanslag, bindestreck, plus	Varning
Boolesk	true/false, ja/nej, 1/0	Tolka → boolean
Text/lista	Fritext (flera värden separerade med komma)	Splitta till array

5.2 Affärsregler

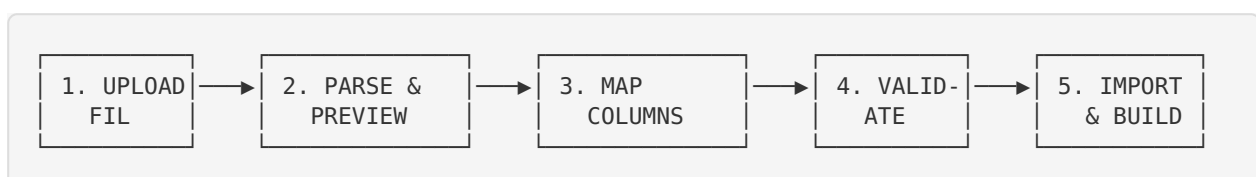
Regel	Beskrivning
<code>Objektnamn</code> obligatoriskt	Varje rad måste ha ett namn
Unik <code>Interimsnummer</code> per objekttyp	Samma interim kan förekomma för butik och dess utrustning
Förälderkonsistens	<code>Interimföräldranummer</code> måste peka på en befintlig rad eller vara tomt
Cirkulärreferens	Föräldrakedjan får inte bilda en loop
<code>Systemnummer</code> existens	Om ifyllt måste det finnas i Traivos databas
Dubbletter	Varning om samma <code>externt_id</code> förekommer mer än en gång

6 Implementeringsspecifikation (Replit)

6.1 Datamodell



6.2 Import-flöde steg-för-steg



Steg 1 - Uppladdning

- Acceptera `.xlsx` och `.csv`
- Skapa en `ImportSession` med status `draft`
- Spara filen i blob storage / filsystem

Steg 2 - Parsing och förhandsgranskning

- Identifiera header-rader (max 3 rader) genom heuristik:
- Rad med flest kända systemfältnamn → systemheader
- Rad med långa beskrivningstexter → beskrivningsrad
- Rad med kortare, unika etiketter → användarrubriker
- Visa kolumnlista med automatiskt föreslagna matchningar
- Visa dataförhandsgranskning (första 10 datarader)

Steg 3 - Kolumnmatchning (UI)

- Visa varje kolumn med:
- Radnummer och kolumnrubrik
- Automatisk matchningsstatus (✓ Matchad / △ Omatchad)
- Knapp "Matcha data +" för manuell koppling
- "Matcha data"-dialog:
- Sökfält för att filtrera tillgängliga fält
- Lista med alla fält grupperade per kategori (standard, adress, kontakt, metadata)
- Varje fält har beskrivning + "Lägg till"-knapp
- Spara mappning i `column_mappings`:

```
json
{
  "A": { "target": "system_id", "type": "standard" },
  "C": { "target": "name", "type": "standard", "required": true },
  "H": { "target": "address.street", "type": "address" },
  "F": { "target": "metadata.typ", "type": "metadata" },
  "Q": { "target": "metadata.antal", "type": "metadata" }
}
```

Steg 4 - Validering

- Iterera över varje datarad:
 1. Kontrollera obligatoriska fält (`name` måste finnas)
 2. Typvalidera varje cell mot mappat fält
 3. Kontrollera referensintegritet:
 - `interim_parent_id` pekar på en befintlig `interim_id` i filen
 - `system_id` (om ifyllt) finns i databasen
 4. Detektera cirkulära föräldrareferenser
 5. Spara valideringsstatus per rad
- Visa sammanfattning:
- Antal giltiga rader
- Antal ogiltiga rader med feldetaljer
- Möjlighet att korrigera/skipa ogiltiga rader

Steg 5 - Import och hierarkiuppbyggnad

```
# Pseudokod för hierarkibyggare
def build_hierarchy(rows, mappings):
    interim_to_object = {} # interim_id → Object UUID
    pending = []           # rader som väntar på sin förälder

    # Steg A: Identifiera befintliga objekt
    for row in rows:
        sys_id = row.get('system_id')
        ext_id = row.get('external_id')
        if sys_id:
            # Matcha mot befintligt objekt → uppdatera
            obj = db.get_object(sys_id)
            interim_to_object[row['interim_id']] = obj.id
        elif ext_id:
            # Försök matcha via extern referens
            obj = db.find_by_external_id(ext_id)
            if obj:
                interim_to_object[row['interim_id']] = obj.id

    # Steg B: Topologisk sortering (förälder före barn)
    sorted_rows = topological_sort(rows, 'interim_id', 'interim_parent_id')

    # Steg C: Skapa/uppdatera objekt i ordning
    for row in sorted_rows:
        parent_id = None
        if row.get('system_parent_id'):
            parent_id = row['system_parent_id']
        elif row.get('interim_parent_id'):
            parent_id = interim_to_object.get(row['interim_parent_id'])

        if row['interim_id'] in interim_to_object:
            # Uppdatera befintligt objekt
            obj = update_object(interim_to_object[row['interim_id']], row, parent_id)
        else:
            # Skapa nytt objekt
            obj = create_object(row, parent_id)
            interim_to_object[row['interim_id']] = obj.id

    # Steg D: Hantera utrustning (delar interim med butik)
    for row in equipment_rows:
        parent_obj_id = interim_to_object.get(row['interim_id'])
        create_object(row, parent_obj_id) # barn till butiken
```

6.3 Matchningsalgoritm (automatisk kolumnmatchning)

```

KNOWN_FIELDS = {
    # Exakta matchningar (case-insensitive)
    'systemnummer': 'system_id',
    'systemföräldranummer': 'system_parent_id',
    'objektnamn': 'name',
    'interimsnummer': 'interim_id',
    'interimföräldranummer': 'interim_parent_id',
    '__empty': '__empty',
    'externt_id': 'external_id',
    'extern_id': 'external_id',
    'leveransadress': 'address.full',
    'delivery latitude': 'position.lat',
    'delivery longitude': 'position.lng',
}

ADDRESS_PATTERNS = {
    'address.gata': 'address.street',
    'address.gatunummer': 'address.street_number',
    'address.postnummer': 'address.postal_code',
    'adress.ort': 'address.city',
}

CONTACT_PATTERNS = {
    'kontaktperson.namn': 'contact.name',
    'kontaktperson.titel': 'contact.title',
    'kontaktperson.telefon': 'contact.phone',
}

def auto_match(column_header):
    normalized = column_header.strip().lower()

    # 1. Exakt matchning
    if normalized in KNOWN_FIELDS:
        return KNOWN_FIELDS[normalized]

    # 2. Adress-/kontaktmönster (punktnotation)
    if normalized in ADDRESS_PATTERNS:
        return ADDRESS_PATTERNS[normalized]
    if normalized in CONTACT_PATTERNS:
        return CONTACT_PATTERNS[normalized]

    # 3. Metadata-mönster
    if normalized.startswith('metadata.'):
        return normalized # behåll som metadata-fält

    # 4. Fuzzy match (Levenshtein ≥ 0.8)
    best = fuzzy_match(normalized, ALL_KNOWN_FIELDS, threshold=0.8)
    if best:
        return best

    # 5. Ej matchad → kräver manuell koppling
    return None

```

6.4 Valideringslogik (detaljerad)

```

VALIDATORS = {
    'text': lambda v: isinstance(v, str) and len(v.strip()) > 0,
    'text_id': lambda v: isinstance(v, (str, int)),
    'integer': lambda v: isinstance(v, (int, float)) and float(v).is_integer(),
    'decimal': lambda v: isinstance(v, (int, float)),
    'email': lambda v: re.match(r'^[^\s@]+@[^\s]+\.[^\s]+$', str(v)),
    'phone': lambda v: re.match(r'^[\d\s\-\+\(\)]+$$', str(v)),
    'date': lambda v: parse_date(v) is not None,
    'boolean': lambda v: str(v).lower() in ('true', 'false', 'ja', 'nej', '1', '0'),
    'gps': lambda v: isinstance(v, (int, float)) and -180 <= float(v) <= 180,
}

FIELD_RULES = {
    'name': { 'type': 'text', 'required': True },
    'system_id': { 'type': 'text_id', 'required': False },
    'system_parent_id': { 'type': 'text_id', 'required': False },
    'interim_id': { 'type': 'text_id', 'required': False },
    'interim_parent_id': { 'type': 'text_id', 'required': False },
    'external_id': { 'type': 'text_id', 'required': False },
    'address.full': { 'type': 'text', 'required': False },
    'address.street': { 'type': 'text', 'required': False },
    'address.postal_code': { 'type': 'text', 'required': False },
    'address.city': { 'type': 'text', 'required': False },
    'position.lat': { 'type': 'gps', 'required': False },
    'position.lng': { 'type': 'gps', 'required': False },
    'contact.name': { 'type': 'text', 'required': False },
    'contact.phone': { 'type': 'phone', 'required': False },
    'contact.email': { 'type': 'email', 'required': False },
}

def validate_row(row, mappings):
    errors = []
    for col, mapping in mappings.items():
        field = mapping['target']
        value = row.get(col)
        rule = FIELD_RULES.get(field, { 'type': 'text', 'required': False })

        if rule['required'] and (value is None or str(value).strip() == ''):
            errors.append(f"Obligatoriskt fält '{field}' saknas")
        elif value is not None and str(value).strip() != '':
            validator = VALIDATORS[rule['type']]
            if not validator(value):
                errors.append(f"Fält '{field}': ogiltigt värde '{value}' (förväntar {rule['type']})")
    return errors

```

6.5 API-endpoints

Metod	Endpoint	Beskrivning
POST	/api/import/upload	Ladda upp Excel/CSV-fil, skapa ImportSession
GET	/api/import/{session_id}	Hämta session med status och preview
GET	/api/import/{session_id}/preview	Hämta kolumner + föreslagna matchningar + datapreview
PUT	/api/import/{session_id}/mappings	Spara/uppdatera kolumn-matchningar
POST	/api/import/{session_id}/validate	Kör validering, returnera resultat
GET	/api/import/{session_id}/validation	Hämta valideringsresultat
POST	/api/import/{session_id}/execute	Starta import (skapa/uppdatera objekt)
GET	/api/import/{session_id}/status	Poll importstatus (progress %)
GET	/api/import/{session_id}/result	Hämta resultat: skapade, uppdaterade, fel
GET	/api/fields	Lista alla tillgängliga systemfält för matchning
GET	/api/fields/search?q={query}	Sök bland tillgängliga fält

Request/response-exempel

POST /api/import/upload

```
// Request: multipart/form-data med filen
// Response:
{
  "session_id": "abc-123",
  "status": "draft",
  "file_name": "Hemköp.xlsx",
  "columns": [
    { "index": 0, "header": "Systemnummer", "user_header": "Finns ej ännu", "auto_match": "system_id", "matched": true },
    { "index": 2, "header": "Objektnamn", "user_header": "Objektnamn", "auto_match": "name", "matched": true },
    { "index": 5, "header": null, "user_header": "Objekt typ", "auto_match": null, "matched": false }
  ],
  "total_rows": 891,
  "preview_rows": [ ... ]
}
```

POST /api/import/{session_id}/execute

```
// Response:
{
  "status": "completed",
  "summary": {
    "total_rows": 891,
    "created": 891,
    "updated": 0,
    "skipped": 0,
    "errors": 0
  },
  "hierarchy": {
    "root_objects": 1,
    "total_levels": 5,
    "total_objects": 891
  }
}
```


6.6 Användarflöde i UI

STEG 1: UPLOAD

Dra och släpp din Excel-/CSV-fil här
eller klicka för att välja fil

[Nästa →]

STEG 2: MATCHA KOLUMNER

1	Referensnummer ⓘ	[extern_t_id]	✓ Matchad
2	Datum ⓘ *	2026-06-08	✓ Matchad
3	Namn ⓘ	[Objektnamn]	✓ Matchad
4	Telefonnummer ⓘ	[Matcha data +]	
5	E-post ⓘ	[Matcha data +]	
6	Leveransadress ⓘ *	[address.full]	✓ Matchad
7	Delivery Latitude ⓘ	[Matcha data +]	
...			
15	Typer ⓘ	[Matcha data +]	
16	Bil	[Matcha data +]	🗑️

[+ Lägg till rubrik]

[- Tillbaka] [Validera →]

(vid klick "Matcha data +")

MATCHA DATA (modal)

Sökrubrik 🔍

Systemnummer
Traivos system..
[Lägg till]

Objektnamn
Obligatoriskt...
[Lägg till]

Interimsnummer
Ditt eget löp...
[Lägg till]

extern_t_id
Metadata-refer...
[Lägg till]

adress.gata
Underfält "gata"..
[Lägg till]

...

STEG 3: VALIDERING

✓ 885 rader giltiga

⚠

6 rader med varningar

✖

0 rader ogiltiga

Varningar:
Rad 145: Telefonnummer saknas
Rad 302: E-post ogiltigt format
...

↶

Tillbaka

Importera

➡

STEG 4: RESULTAT

✅

 Import klar!

Skapade: 891 objekt
Uppdaterade: 0 objekt
Hoppade: 0 rader

Hierarki:

☐

Axfood Sverige

- ☐

Hemköp kedjan
 - ☐

Göteborg Egenägd (28 butiker)
 - ☐

Mitt Egenägd (22 butiker)
 - ☐

...

[Visa alla objekt] [Ny import]

7 Datastatistik (Hemköp-filen)

Mått	Värde
Totalt datarader	891
Organisation/kund	1 (Axfood Sverige)
Kedja	1 (Hemköp kedjan)
Distrikt	10
Butiker	220
Utrustning totalt	659
– Pantkär	220
– Matavfallskär	220
– Kundvagnsgarage	219
Hierarkinivåer	5
Kolumner med data	19
Systemfält matchade	11
Metadatakolumner	4 (Objekt typ, Antal, Tömningsdag, Färg)
Adresskolumner	4 (gata, gatunr, postnr, ort)
Kontaktkolumner	3 (namn, titel, telefon) + e-post

8 Teknisk stack (rekommendation för Replit)

Komponent	Teknik
Backend	Node.js + Express / Python + FastAPI
Databas	PostgreSQL med JSONB-stöd
Filparsning	<code>xlsx</code> (Node) / <code>openpyxl</code> (Python)
Validering	Zod (Node) / Pydantic (Python)
Frontend	React + Tailwind CSS
Filuppladdning	Multer (Node) / python-multipart
State management	React Query / SWR
Hierarkivisualisering	react-d3-tree eller liknande

9 Beslutstabell (matchning av objekt)

A. Finns systemnummer?	→ JA → Uppdatera befintligt objekt → NEJ → Gå till B
B. Finns <code>externt_id</code> ? Hittas?	→ JA → Sök i databasen → JA → Uppdatera befintligt objekt → NEJ → Gå till C
C. Finns <code>interimsnummer</code> ? Finns <code>interimförälder</code> ?	→ JA → Skapa nytt objekt med interim-ID → JA → Koppla till föräldern → NEJ → Placera på rotnivå / klustra
D. Fel datatyp eller obligatoriskt fält saknas?	→ JA → Stoppa raden, rapportera → för korrigering

10 Punktnotation (sammansatta fält)

Kolumner med punktnotation (t.ex. `adress.gata` , `kontaktperson.namn`) grupperas automatiskt till sammansatta JSON-objekt:

```
// Input-kolumner:  
//   adress.gata = "Björnövägen"  
//   adress.gatunummer = "10"  
//   adress.postnummer = "151 65"  
//   adress.ort = "Södertälje"  
  
// Resulterar i:  
{  
  "address": {  
    "street": "Björnövägen",  
    "street_number": "10",  
    "postal_code": "151 65",  
    "city": "Södertälje"  
  }  
}
```

Detta gäller alla fält med punkt i namnet – systemet splittar på `.` och bygger nästlat objekt.

Specifikation skapad 2026-06-08 baserat på Hemköp-importfil och Traivo UI-skärmdumpar.